

整体结构

```
root@6a687ab5834b:/workspace/workspace/cmodel/cmodel# ll
total 2504
drwxr-xr-x 11 root root    4096 Mar 27 08:25 ./
drwxr-xr-x  4 root root    4096 Mar 20 09:36 ../
drwxr-xr-x  8 root root    4096 Mar 27 08:24 .git/
-rw-r--r--  1 root root     31 Mar 27 06:08 .gitignore
drwxr-xr-x  2 root root    4096 Mar 27 06:08 .vscode/
-rw-r--r--  1 root root   1050 Mar 27 07:34 Makefile
-rw-r--r--  1 root root    668 Mar 27 09:14 README.md
drwxr-xr-x  2 root root    4096 Mar 27 08:24 booth/
drwxr-xr-x  2 root root   12288 Mar 27 08:25 build/
-rwxr-xr-x  1 root root    5190 Mar 27 07:34 command_script_template.tcl*
drwxr-xr-x  2 root root    4096 Mar 27 08:24 common/
drwxr-xr-x  2 root root    4096 Mar 27 08:24 dop/
drwxr-xr-x  5 root root    4096 Mar 27 08:24 sfu/
drwxr-xr-x  2 root root   12288 Mar 27 06:09 softfloat/
-rw-r--r--  1 root root  1401576 Mar 27 08:25 softfp.a
-rwxr-xr-x  1 root root 1073584 Mar 27 08:25 test.elf*
drwxr-xr-x  2 root root    4096 Mar 27 07:34 tests/
```

验证代码正确性

下载下来的 comdel 代码整体结构如上所示。

下载完的代码请在 linux 系统下尝试编译，看是否能正常编译完成。

如按照下列步骤操作正常完成，则表示代码正确。如出现任何错误，则请更新代码再尝试编译。

1. 清除前期编译环境：

```
/cmodel# make distclean
```

2. 编译整个 cmodel：

```
/cmodel# make USE_EXTERN_C=1
```

3. 用 main.cpp 进行测试：

```
/cmodel# ./test.elf
```

准备 DPV 环境

1. 按照下面的格式存放 cmodel 代码：

所有 cmodel 代码，不改变代码结构。

```
lilia@cd-login03#lilia] : /projects/platform/temp/lilia/example1/ > ls
DPV_FMA_tutorial.pdf  Hclang          cmodel          novas.rc        run             vcf.log         vcst_command.log  vcst_rtdb.bak
Design.pdf            README_VCFormal_DPV_Advanced.pdf  novas.conf      rtl             tcl             vcf.log.bak     vcst_rtdb        verdilog
```

2. 在 cmodel 文件夹下增加 DPV 工具的入口函数文件，需自行增加：

cmodel 支持的功能到 common/cmode.h 里去查看函数接口。

比如待验证的 rcp 功能：

#include "cmodel.h" 为 cmodel 的调用入口，sfu_rcp_fp32() 为 cmodel.h 定义好的功能函数，要测试什么功能在 cpp 里面增加

```
sfu_rcp_zc.cpp /projects/platform/temp/lilia/example1/cmodel) -
File Edit Tools Syntax Buffers Window Help
#include "Hector.h"
#include "cmodel.h"

4
5
6 void hector_wrapper()
7 {
8     uint8_t float_rounding_mode = round_to_even;
9     uint32_t x, result;
10
11     Hector::registerInput("x", &x, 8 * sizeof(x));
12     Hector::registerOutput("result", &result, 8 * sizeof(result));
13
14     Hector::beginCapture();
15
16     result = sfu_rcp_fp32(x, float_rounding_mode);
17
18     Hector::endCapture();
19 }
20 }
```

3. 如在 tcl 文件夹下开启 DPV 工具，则将 cmodel/command_script_template.tcl 拷贝到 tcl/下进行修改，自行增加其他部分：

注意不要修改如下部分内容：

```
1 set _hector_comp_use_new_flow true
2 set _hector_softfloat_version custom
3
4 set_fml_appmode DPV
5 set_hector_sym_maxstackdepth 200
6 set_hector_sym_maxiter_partial 400
7
```

在 make 命令这里可以选择编译全部的代码 compile_spec，也可以选择只编译 sfu 模块，如上测试 rcp 则可选则 compile_spec_sfu。如果测试 dop 模块，则可以选择

comiple_spec_dop.

```
proc make {} {  
    compile spec  
    #compile_impl  
    #compose  
}
```

```
proc make {} {  
    compile spec sfu  
    #compile_impl  
    #compose  
}
```

4. 运行 DPV 工具，在工具内编译 cmodel 代码 (tcl 脚本改为自己的脚本名) :

```
tcl/ > bsub -Is vcf -verdi -f command_script_rcp_zc.tcl &
```

vcf> make

等待编译完成，成功。

VC Formal Console

Rule View Tools Window Help

Line: 12542

```
12542 Info CC-003: Creating formal information ... success (368 files)  
12543 Info CC-004: Linking formal information (stage 1) ... success  
12544 Info CC-005: Linking formal information (stage 2) ... success  
12545 Constructing DFG for hector_wrapper()  
12546 Simulating global initialization __init_0... ok  
12547 Registering input 'x' with bitwidth 32.  
12548 Registering output 'result' with bitwidth 32.  
12549 Elapsed time: 1.65s user, 0s system, 1.64s real.  
12550 Simulated 4346 statements in 1.65s, equivalent 2633 statements per second.  
12551 Final number of DFG nodes: 3403  
12552 INFO: Writing DFG to file "spec.dfg".  
12553 INFO: Finished compile_design spec (23 sec)  
12554 INFO: Number of DFG nodes : 3403  
12555 compile_design spec done
```